

Reviewer Pack — Minimal Lifting Index Correlation with SAT Clause Set Entrop...

Entry f3d3447db1f1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 02:18:30 UTC

Minimal Lifting Index Correlation with SAT Clause Set Entropy

Entry ID: f3d3447db1f1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 02:18:30 UTC

1. Conjecture

Field A (mathematical branch): Lifting Theory **Field B** (complexity object): SAT Clause Sets

Statement:

For all conjunctive normal forms (CNFs) φ , the minimal lifting index ($\text{mli}(\varphi)$) is logarithmically correlated with the entropy of φ 's clause set ($\text{ent}(\varphi)$), such that $\text{mli}(\varphi) = \Theta(\log \text{ent}(\varphi))$.

Rationale (proposer's reasoning):

Lifting theory studies the invariants of representations, and its lifting index measures the complexity of a representation. Correlating this with the entropy of SAT clause sets could reveal structural relationships between representational complexity and the informational content of problem instances.

Taxonomy category: lifting_theory_sats_clauses (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 997b99a57a6b241b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all CNFs φ with n variables ($n \leq 40$), the correlation coefficient between $\text{mli}(\varphi)$ and $\log \text{ent}(\varphi)$ is ≥ 0.8 across at least three seeds, and the mean absolute difference between $\text{mli}(\varphi)$ and $\Theta(\log \text{ent}(\varphi))$ is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Helper functions
def log2(x):
    return math.log2(x) if x > 0 else -math.inf

def entropy(probs):
    return sum(p * log2(p) for p in probs if p > 0)

def gaussian_elimination(A, b):
    n = len(b)
    M = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(M[r][i]))
        M[i], M[max_row] = M[max_row], M[i]
        if M[i][i] == 0:
            raise ValueError("Singular matrix")
        for j in range(i + 1, n):
            factor = M[j][i] / M[i][i]
            for k in range(n + 1):
                M[j][k] -= factor * M[i][k]
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (M[i][n] - sum(M[i][j] * x[j] for j in range(i + 1, n))) / M[i][i]
    return x

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Parameters
    n_values = [5, 10, 15, 20, 30, 40]
    instances_per_n = 30
    total_instances = sum(instances_per_n for n in n_values)

    mli_sum = 0
    log_ent_sum = 0
    instances_tested = 0
    n_max = 1
```

```

# Generate random CNFs and compute mli and ent
for n in n_values:
    for _ in range(instances_per_n):
        clauses = []
        for _ in range(random.randint(1, n * (n + 1) // 2)):
            clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(random.randint(1, n))]
            if all(clause[i] != -clauses[j][i] for j in range(len(clauses))):
                clauses.append(clause)
        mli = len(clauses) / (n * (n + 1) // 2)
        ent = entropy([len(clause) / n for clause in clauses])
        mli_sum += mli
        log_ent_sum += log2(ent)
        instances_tested += 1
        n_max = max(n_max, n)

# Compute correlation and mean absolute difference
if instances_tested == total_instances:
    avg_mli = mli_sum / instances_tested
    avg_log_ent = log_ent_sum / instances_tested
    correlation = (mli_sum * log_ent_sum - instances_tested * avg_mli * avg_log_ent) / (
        math.sqrt(mli_sum**2 - instances_tested * avg_mli**2) *
        math.sqrt(log_ent_sum**2 - instances_tested * avg_log_ent**2)
    )
    mean_abs_diff = abs(avg_mli - avg_log_ent)

# Check acceptance criterion
if correlation >= 0.8 and mean_abs_diff <= 3:
    conjecture_holds = True
    counterexample = ""
else:
    conjecture_holds = False
    counterexample = "correlation_threshold_not_met"

else:
    conjecture_holds = False
    counterexample = "insufficient_instances"

return {
    "metric_name": "mli_vs_log_ent",
    "metric_value": avg_mli,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

```

```

# Compute mean and std of metric_value
if all("metric_value" in r for r in results):
    values = [r["metric_value"] for r in results]
    mean_value = sum(values) / len(values)
    std_value = math.sqrt(sum((v - mean_value)**2 for v in values) / len(values))

# Compute fraction of seeds where conjecture_holds
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(r["counterexample"] == "correlation_threshold_not_met" for r in results):
    print("RESULT: FALSIFIED counterexample=\"correlation_threshold_not_met\" first_failing_seed=")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
else:
    print("RESULT: INCONCLUSIVE missing_metric_value")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_708904ad.py", line 106, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_708904ad.py", line 61, in run_trial
    if all(clause[i] != -clauses[j][i] for j in range(len(clauses))):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_708904ad.py", line 61, in <genexpr>
    if all(clause[i] != -clauses[j][i] for j in range(len(clauses))):
           ^
NameError: name 'i' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'i', which prevented the production of data necessary to evaluate the conjecture. | next: Review and fix the error in the test code, then rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14720
2	preregistration	ollama_remote	glm4:latest	0	0	9373
3	novelty	ollama_remote	glm4:latest	0	0	15809
4	novelty	ollama_remote	glm4:latest	0	0	13243
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12883
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15457
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12437
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14750
9	judge	ollama_remote	glm4:latest	0	0	11723

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120394 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f3d3447db1f1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f3d3447db1f1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f3d3447db1f1.tar.gz` (if generated)